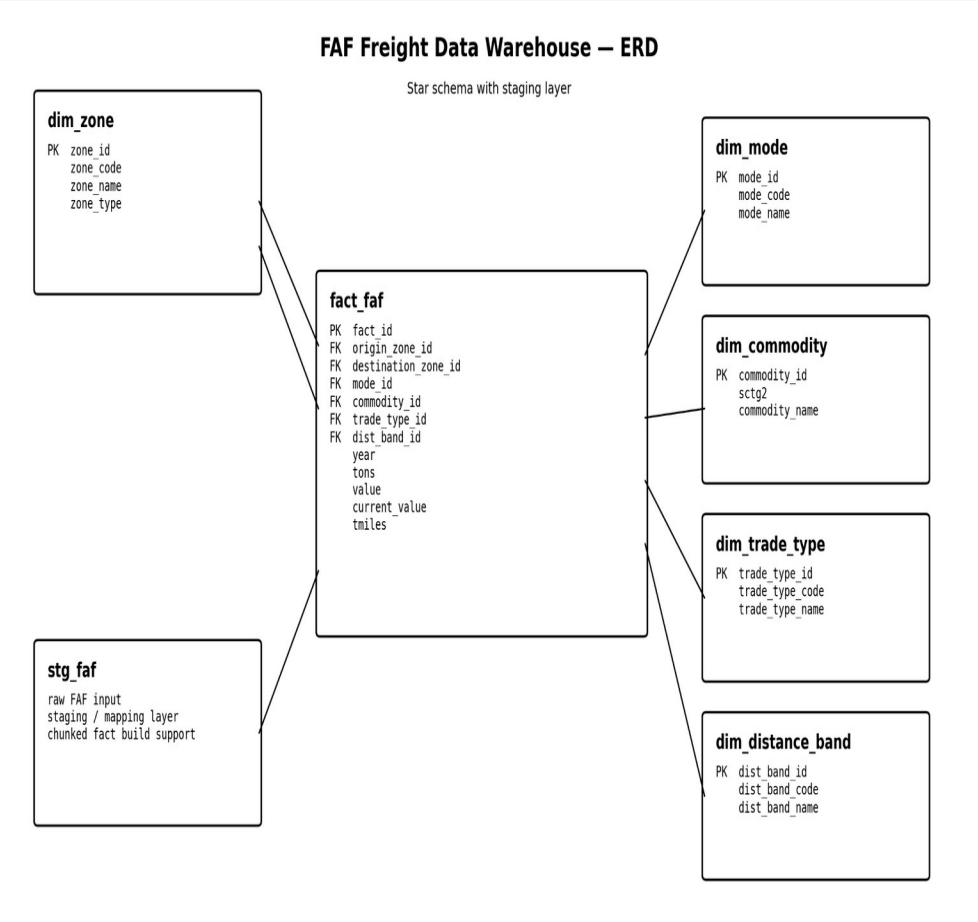
FAF Freight Data Warehouse

Building an analytics-ready logistics warehouse from raw

Artur Melnyk
Applied Data Science / Analytics Engineering Portfolio Project
Python • MySQL • Pandas • SQLAlchemy • Star-schema modeling
Dataset: Freight Analysis Framework (FAF)



Problem

Freight datasets are large, wide, and difficult to analyze directly

Measures are encoded per year, e.g. tons_YYYY and value_YYYY

The raw structure is not optimized for repeated analytical questions

Without indexing and warehouse design, queries become slow and expensive

Goal: convert raw FAF data into a validated, reproducible analytical warehouse

Dataset

Source & Scale

Freight Analysis Framework (FAF 5.7.1)

U.S. Bureau of Transportation Statistics

~600+ MB CSV

~2.5 million freight records

Years covered: 2018–2024

Measures & Structure

Measures: tons, value, current_value, tmiles

Wide source format with annual suffixes

Requires wide → long transformation before warehousing

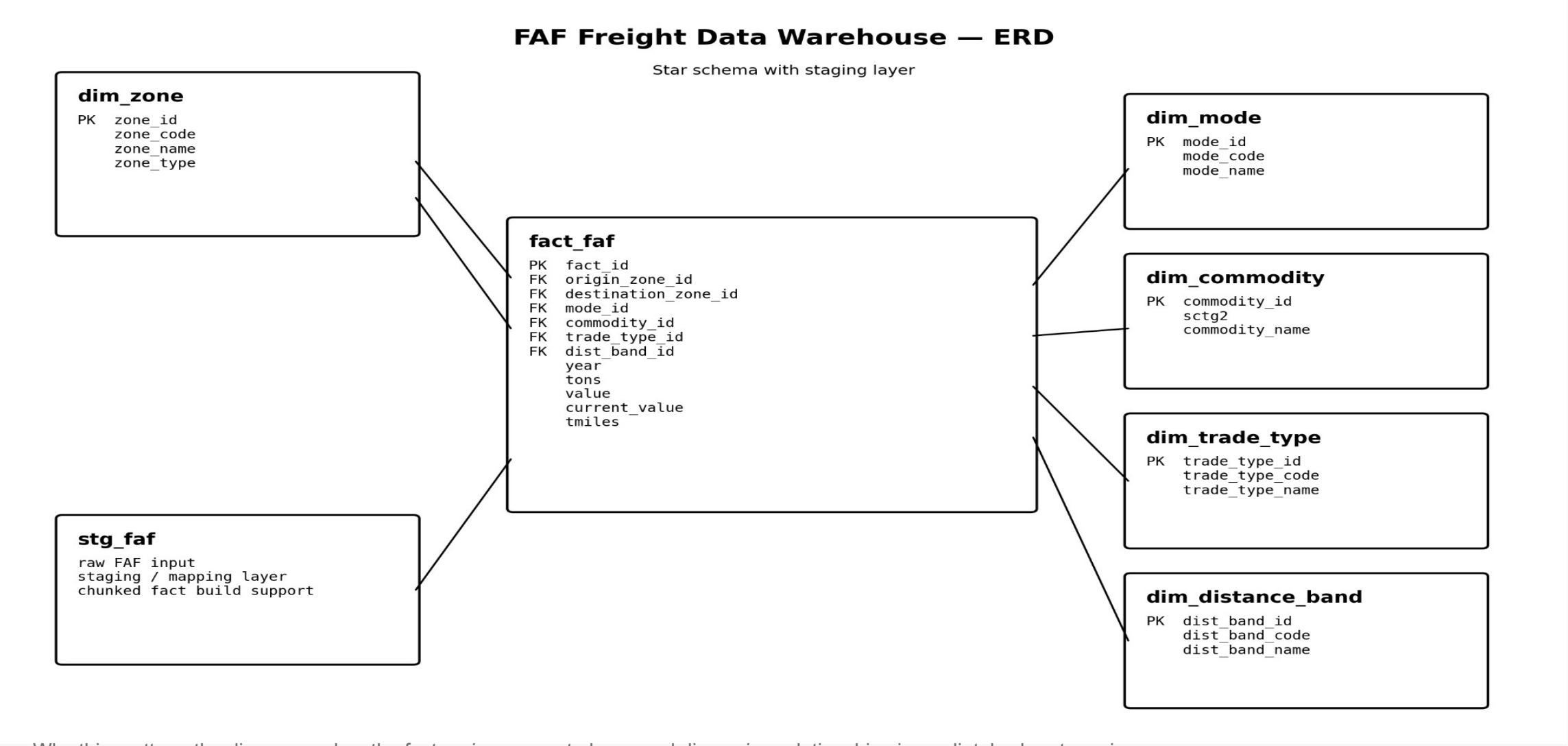
Well-suited for dimensional modeling after profiling

Data Profiling

- Chunk-based scans were used to profile the full dataset safely
- Checks included null analysis, duplicate detection, negative measures, and structural sampling
- Profiling identified the analytical grain, key entities, measures, and dimensional candidates
- Result: the dataset was structurally consistent and suitable for dimensional modeling

Warehouse Architecture (ERD)

Star schema centered on fact_faf with staging and dimensions



Why this matters: the diagram makes the fact grain, surrogate keys, and dimension relationships immediately clear to reviewers.

Warehouse Design

Core Model

Central fact table: fact_faf

Dimensions: dim_zone, dim_mode, dim_commodity, dim_trade_type, dim_distance_band

Staging layer: stg_faf

Fact Grain

One row per origin, destination, mode, commodity, trade type, distance band, and year

Supports clean aggregation

Ensures additive measures and stable analytical grain

ETL Pipeline

1. Build dimensions
2. Load raw records into staging
3. Transform wide data into long format
4. Build the fact table with mapped surrogate keys
5. Run validation checks
6. Apply performance improvements
7. Execute analytics queries

Data Validation

Validation Checks

- Orphan foreign keys
- Duplicate fact grain
- Null critical keys
- Negative measures
- Staging consistency checks

Why It Matters

- Protects referential integrity
- Prevents silent analytical errors
- Improves trust in dashboard and query outputs
- Makes the warehouse reproducible and reliable

Performance Engineering

Problem

Year-filtered analytical queries were initially very slow

Top lanes by value: ~128 seconds

Mode + commodity mix: ~131 seconds

Measured Improvement

Composite indexes aligned with WHERE + GROUP BY patterns

Top lanes query: 128s → 2.6s

Mode + commodity query: 131s → 0.94s

Overall improvements: ~50× to ~170×

Analytics Layer

Top freight corridors

Commodity concentration analysis

Transportation mode mix

Trade type distribution

Distance-band analysis

Year-over-year freight changes

Zone inflow / outflow balance

Example Business Insights

Where is freight value concentrated?

Which commodities dominate shipments?

How is freight distributed across transportation modes?

Is the network becoming more local or more long-haul?

Which zones are net exporters versus net importers?

These outputs support logistics planning, infrastructure decisions, and freight network monitoring

Engineering Outcomes

Dimensional modeling

SQL warehouse design

ETL pipeline design

Data validation engineering

Performance optimization

Decision-ready analytics

Final Takeaway

Raw data → Profiling → Dimensional modeling → ETL → Validation → Performance tuning → Business analytics

This project demonstrates a complete analytics engineering workflow

The result is a reproducible freight analytics warehouse ready for decision support